

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn

!wget "https://drive.google.com/uc?export=download&id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU" -O healthyfime.csv

--2024-04-19 16:35:14-- https://drive.google.com/uc?export=download&id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU
Resolving drive.google.com (drive.google.com)... 142.250.103.101, 142.250.103.102, 142.250.103.139, ...
Connecting to drive.google.com (drive.google.com)|142.250.103.101|:443... connected.
HTTP request sent, awaiting response... 303 See Other
Location: https://drive.usercontent.google.com/download?id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU&export=download [following]
--2024-04-19 16:35:14-- https://drive.usercontent.google.com/download?id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU&export=down
Resolving drive.usercontent.google.com (drive.usercontent.google.com)... 172.217.219.132, 2607:f8b0:4001:c13::84
Connecting to drive.usercontent.google.com (drive.usercontent.google.com)|172.217.219.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 761835 (744K) [application/octet-stream]
Saving to: 'healthyfime.csv'

healthyfime.csv      100%[=====>] 743.98K  --.-KB/s    in 0.01s

2024-04-19 16:35:15 (75.7 MB/s) - 'healthyfime.csv' saved [761835/761835]
```

```
!wget "https://drive.google.com/uc?export=download&id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU" -O healthyfime.csv

--2024-04-19 16:35:16-- https://drive.google.com/uc?export=download&id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU
Resolving drive.google.com (drive.google.com)... 209.85.234.113, 209.85.234.100, 209.85.234.101, ...
Connecting to drive.google.com (drive.google.com)|209.85.234.113|:443... connected.
HTTP request sent, awaiting response... 303 See Other
Location: https://drive.usercontent.google.com/download?id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU&export=download [following]
--2024-04-19 16:35:16-- https://drive.usercontent.google.com/download?id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU&export=down
Resolving drive.usercontent.google.com (drive.usercontent.google.com)... 108.177.121.132, 2607:f8b0:4001:c19::84
Connecting to drive.usercontent.google.com (drive.usercontent.google.com)|108.177.121.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 761835 (744K) [application/octet-stream]
Saving to: 'healthyfime.csv'

healthyfime.csv      100%[=====>] 743.98K  --.-KB/s    in 0.02s

2024-04-19 16:35:17 (40.1 MB/s) - 'healthyfime.csv' saved [761835/761835]
```

```
df= pd.read_csv('healthyfime.csv')
df.head()
```

	age	gender	height_cm	weight_kg	body fat_%	diastolic	systolic	gripForce	foi
0	27.0	M	172.3	75.24	21.3	80.0	130.0	54.9	
1	25.0	M	165.0	55.80	15.7	77.0	126.0	36.4	
2	31.0	M	179.6	78.00	20.1	92.0	152.0	44.8	
3	32.0	M	174.5	71.10	18.4	76.0	147.0	41.4	
4	28.0	M	173.8	67.70	17.1	70.0	127.0	43.5	

Next steps: [View recommended plots](#)

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13393 entries, 0 to 13392
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   age                                    13393 non-null  float64
1   gender                                13393 non-null  object
2   height_cm                             13393 non-null  float64
3   weight_kg                             13393 non-null  float64
4   body fat_%                            13393 non-null  float64
5   diastolic                             13393 non-null  float64
6   systolic                              13393 non-null  float64
7   gripForce                             13393 non-null  float64
8   sit and bend forward_cm               13393 non-null  float64
9   sit-ups counts                        13393 non-null  float64
10  broad jump_cm                         13393 non-null  float64
```

```
11 class 13393 non-null object
dtypes: float64(10), object(2)
memory usage: 1.2+ MB
```

```
df.replace({"M":0, "F":1} , inplace = True)
df.head()
```

	age	gender	height_cm	weight_kg	body fat_%	diastolic	systolic	gripForce	foi
0	27.0	0	172.3	75.24	21.3	80.0	130.0	54.9	
1	25.0	0	165.0	55.80	15.7	77.0	126.0	36.4	
2	31.0	0	179.6	78.00	20.1	92.0	152.0	44.8	
3	32.0	0	174.5	71.10	18.4	76.0	147.0	41.4	
4	28.0	0	173.8	67.70	17.1	70.0	127.0	43.5	

Next steps:

[View recommended plots](#)

```
classes = list(df['class'].unique())
mapping_dict = { ch : i for i, ch in enumerate(sorted(classes, reverse=True)) }
print (mapping_dict)
df['class'].replace(mapping_dict , inplace = True)
df.head()
```

```
{'D': 0, 'C': 1, 'B': 2, 'A': 3}
```

	age	gender	height_cm	weight_kg	body fat_%	diastolic	systolic	gripForce	foi
0	27.0	0	172.3	75.24	21.3	80.0	130.0	54.9	
1	25.0	0	165.0	55.80	15.7	77.0	126.0	36.4	
2	31.0	0	179.6	78.00	20.1	92.0	152.0	44.8	
3	32.0	0	174.5	71.10	18.4	76.0	147.0	41.4	
4	28.0	0	173.8	67.70	17.1	70.0	127.0	43.5	

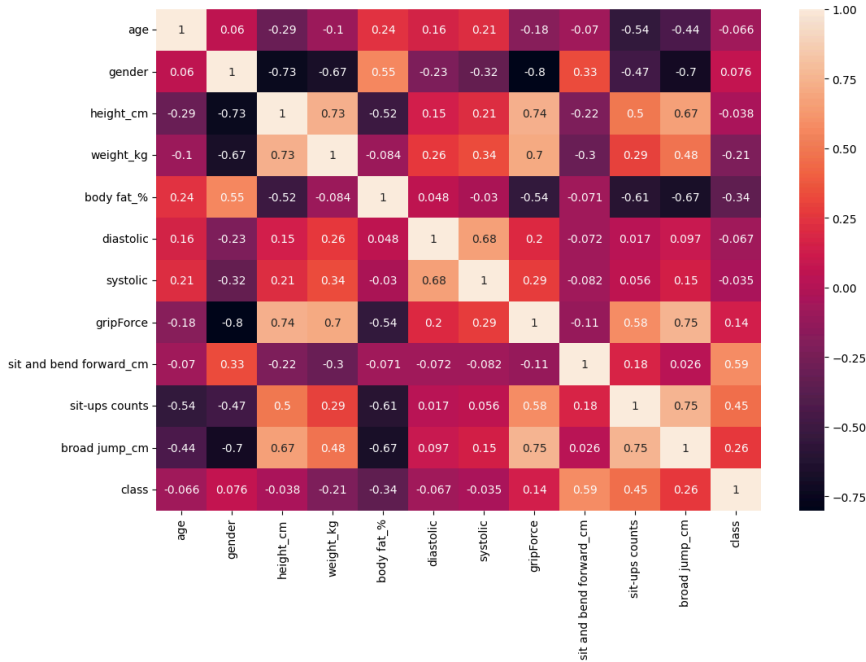
Next steps:

[View recommended plots](#)

```
plt.figure(figsize=(12,8))

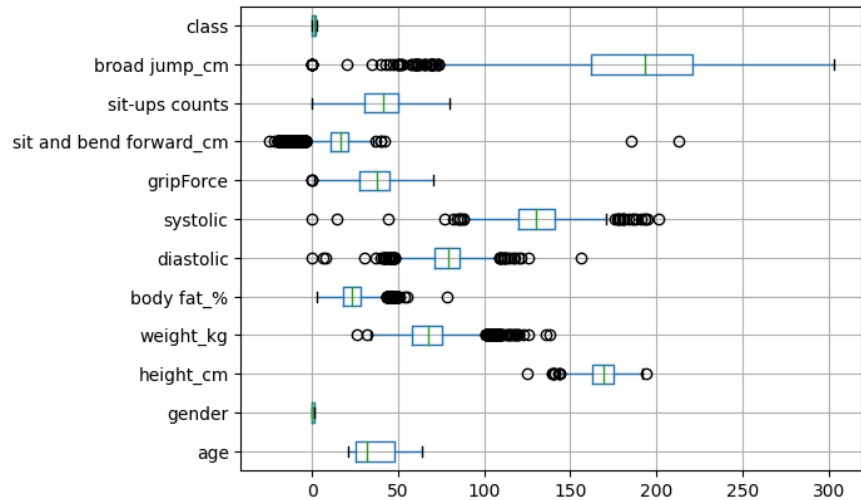
sns.heatmap(df.corr(), annot=True)
```

<Axes: >



```
df.boxplot(rot = 0, vert=False)
```

<Axes: >



```
X, y = df.iloc[:, :-1], df.iloc[:, -1]
print(X.shape, y.shape)
```

(13393, 11) (13393,)

```
from sklearn.model_selection import train_test_split

X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.1, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.1, random_state=42)

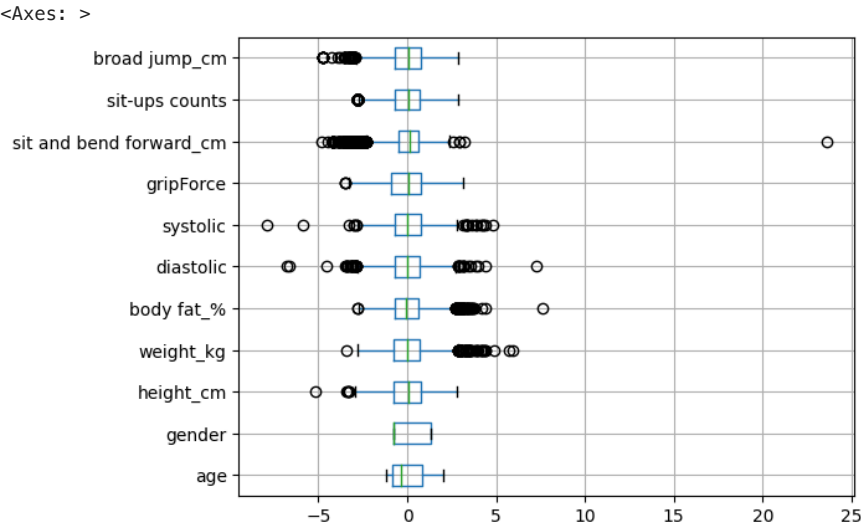
print('Train : ', X_train.shape, y_train.shape)
print('Valid : ', X_val.shape, y_val.shape)
print('Test : ', X_test.shape, y_test.shape)

Train : (10847, 11) (10847,)
Valid : (1206, 11) (1206,)
Test : (1340, 11) (1340,)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train,y_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

pd.DataFrame(X_train,columns=df.columns[:-1]).boxplot(rot = 0, vert=False)
```



```
import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([Dense(64, activation='relu', input_shape=(11,)),
                    Dense(4, activation='Softmax')
                    ])

for i in model.weights:
    print(i.shape)

(11, 64)
(64,)
(64, 4)
(4,)
```

```
model.summary()
```

```
Model: "sequential"
Layer (type)                Output Shape                Param #
=====
dense (Dense)                (None, 64)                 768
dense_1 (Dense)              (None, 4)                 260
=====
Total params: 1028 (4.02 KB)
Trainable params: 1028 (4.02 KB)
Non-trainable params: 0 (0.00 Byte)
```

19/04/2024, 23:33Tensorflow Callbacks - Colab

```
model = Sequential([Dense(64, activation='relu', input_shape=(11,),name="Hidden1"),
                    Dense (4, activation='Softmax', name='Output')
                    ])

model.summary()

Model: "sequential_1"
-----
Layer (type)                Output Shape                Param #
-----
Hidden1 (Dense)              (None, 64)                  768
Output (Dense)                (None, 4)                   260
-----
Total params: 1028 (4.02 KB)
Trainable params: 1028 (4.02 KB)
Non-trainable params: 0 (0.00 Byte)
-----

model.compile(
    loss="sparse_categorical_crossentropy",
    metrics=['accuracy'],
    #optimizer='adam'
    #
    #optimizer=tf.keras.optimizers.Adam(learning_rate=0.01)
    optimizer = tf.keras.optimizers.SGD(momentum=0.9),
)

class VerboseCallback(tf.keras.callbacks.Callback):
    # runs only before the training starts
    def on_train_begin(self, logs=None):
        print("Starting training...")

    # runs after every epoch
    def on_epoch_end(self, epoch, logs = None):
        if epoch % 50 == 0:
            print(f'Epoch {str(epoch).zfill(3)}', '- loss : ', logs['loss'], '- Acc : ', logs['accuracy'])

    # runs once training is finished
    def on_train_end(self, logs=None):
        print("...Finished training")

#https://www.tensorflow.org/guide/keras/writing_your_own_callbacks

history = model.fit(X_train, y_train, epochs=500, batch_size=256, validation_split=0.1, verbose=0, callbacks=[VerboseCallba

Starting training...
Epoch 000 - loss : 0.9888563752174377 - Acc : 0.5540872812271118
Epoch 050 - loss : 0.5917516946792603 - Acc : 0.7546609044075012
Epoch 100 - loss : 0.571320116519928 - Acc : 0.7596803903579712
Epoch 150 - loss : 0.5588457584381104 - Acc : 0.7717680931091309
Epoch 200 - loss : 0.5504686236381531 - Acc : 0.771665632724762
Epoch 250 - loss : 0.5426731705665588 - Acc : 0.7750461101531982
Epoch 300 - loss : 0.5386110544204712 - Acc : 0.7774021625518799
Epoch 350 - loss : 0.5348998308181763 - Acc : 0.777094841003418
Epoch 400 - loss : 0.5386397838592529 - Acc : 0.7791436314582825
Epoch 450 - loss : 0.5359981656074524 - Acc : 0.7772997617721558
...Finished training

# Tensorboard UI

#pip install tensorboard
#conda install -c conda-forge tensorboard

%load_ext tensorboard

log_flow='logs'

!rm -rf logs

from tensorflow.keras.callbacks import TensorBoard
```

```
tb_callback= TensorBoard(log_dir='logs', histogram_freq=1)

history = model.fit(X_train, y_train, epochs=500, batch_size=256, validation_split=0.1, verbose=0,
                    callbacks=[VerboseCallback(),tb_callback])

Starting training...
Epoch 000 - loss : 0.5384678244590759 - Acc : 0.7758656144142151
Epoch 050 - loss : 0.5310496687889099 - Acc : 0.7797582745552063
Epoch 100 - loss : 0.5303385853767395 - Acc : 0.7814996838569641
Epoch 150 - loss : 0.5301957726478577 - Acc : 0.7833435535430908
Epoch 200 - loss : 0.5337632894515991 - Acc : 0.7788363099098206
Epoch 250 - loss : 0.526850700378418 - Acc : 0.7801679968833923
Epoch 300 - loss : 0.5283268094062805 - Acc : 0.782524049282074
Epoch 350 - loss : 0.525139570236206 - Acc : 0.7828313708305359
Epoch 400 - loss : 0.5260028839111328 - Acc : 0.7811923623085022
Epoch 450 - loss : 0.5331113934516907 - Acc : 0.777094841003418
...Finished training

%tensorboard --logdir={'logs'}
```

